

Compilador MiniJava

Etapa 2

Implementación del Análisis Sintáctico

Compiladores e Intérpretes
Septiembre de 2025

Ludmila Prolygin

Tabla de contenidos	
Compilación y ejecución del código fuente	2
Decisiones de diseño	2
Clases	2
Paquetes	2
Consideraciones adicionales	2
Gramática	3
Logros	6
Logros de la etapa 1	6
Logros de la etapa 2	6
Repositorio	7

Compilación y ejecución del código fuente

Para realizar la compilación del código fuente entregado, se utiliza el siguiente comando por consola Linux:

```
javac -sourcepath src -d out src/main/Main.java
```

En el caso de la ejecución, para invocar el programa, se utiliza

```
java -cp out main.Main <archivo de test>
```

Se incluye en el [repositorio](#) un archivo .jar que puede ser ejecutado con

```
java -jar Compilador.java <fuente.ext>
```

Decisiones de diseño

Clases

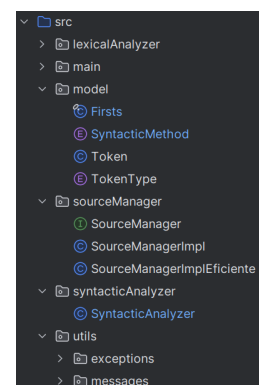
- `LexicalAnalyzer`. Esta clase se encarga de manejar el pasaje de estados de los autómatas que describen el comportamiento del lenguaje.
- `TokenType`. Enumerado que identifica los diferentes tipos de tokens disponibles en el lenguaje. Cada uno de ellos se vincula con una cadena de caracteres descriptiva del tipo.
- `LexicalException`. Modela las excepciones léxicas.
- `SyntacticMethod`. Enumerado que identifica las diferentes partes izquierdas de las reglas sintácticas.
- `Firsts`. Mapeo que permite llevar registro de los primeros de cada regla sintáctica de la gramática.

Paquetes

- `main`. Incluye todas las clases ejecutables.
- `model`. Contiene a las clases `Token` y `TokenType` que describen los terminales del lenguaje.
- `sourceManager`. La interfaz y clase que la implementa, provistas por la cátedra, están incluidas en este paquete.

Consideraciones adicionales

- Aquellos métodos nombrados con `_` (guión bajo) en el comienzo de su identificador refieren a aquellas reglas no pertenecientes a la gramática original propuesta; ya sea por factorización o realización de opcionales.
- El orden de las instrucciones del método `noTerminales` de la clase `Firsts` no es azaroso; se tuvieron en cuenta las dependencias de las reglas para poder calcular los *primeros* correctamente.



Gramática

```
<Inicial> ::= <ListaClases> EOF
<ListaClases> ::= <Clase><ListaClases> | <_Interface><ListaClases> | ε
<_Interface> ::= <ModificadorOpcional> interface idClase
    <_TipoParametricoOpcional> <_OptionalParent>
    { <_ComportamientoInterface> }
<_ComportamientoInterface> ::= <_SignaturaMetodo>
    <_ComportamientoInterface> | ε
<_SignaturaMetodo> ::= <_VisibilidadOpcional> <TipoMetodo>
    <_TipoParametricoOpcional> idMetVar <ArgsFormales> ;
<Clase> ::= <ModificadorOpcional> class idClase <_TipoParametricoOpcional>
    <_OptionalParent> { <ListaMiembros> }
<_OptionalParent> ::= <HerenciaOpcional> | <_InterfaceOpcional> | ε
<_TipoParametricoOpcional> ::= < idClase > | ε
<ModificadorOpcional> ::= abstract | static | final | ε
<HerenciaOpcional> ::= extends idClase <_TipoParametricoOpcional> | ε
<_InterfaceOpcional> ::= implements idClase <_TipoParametricoOpcional> | ε
<ListaMiembros> ::= <_MiembroCompleto> <ListaMiembros> | ε
<_MiembroCompleto> ::= <_VisibilidadOpcional> <Miembro>
<_VisibilidadOpcional> ::= public | private | ε
<Miembro> ::= <_DecisorMiembroTipo> |
    void idMetVar <ArgsFormales> <BloqueOpcional> |
    <_Modificador> <TipoMetodo> <_TipoParametricoOpcional> idMetVar
    <ArgsFormales> <BloqueOpcional>
<_DecisorMiembroTipo> ::= idClase <_DecisorMiembroIdClase> |
    <TipoPrimitivo><_TipoParametricoOpcional> idMetVar <_RestoMiembro>
<_DecisorMiembroIdClase> ::= <ArgsFormales> <Bloque> |
    <_TipoParametricoOpcional> idMetVar <_RestoMiembro>
<_RestoMiembro> ::= ; | <_InicializacionAtributoOpcional> ; |
    <ArgsFormales> <BloqueOpcional>
<_InicializacionAtributoOpcional> ::=
    = <ExpresionCompuesta> <_OperadorTernario> | ε
<_Modificador> ::= static | abstract | final
<TipoMetodo> ::= <Tipo> | void
<Tipo> ::= <TipoPrimitivo> | idClase
<TipoPrimitivo> ::= boolean | char | int
```

```

<ArgsFormales> ::= ( <ListaArgsFormalesOpcional> )
<ListaArgsFormalesOpcional> ::= <ListaArgsFormales> | ε
<ListaArgsFormales> ::= <ArgFormal> <_RestoListaArgsFormales>
<_RestoListaArgsFormales> ::= , <ArgFormal> <_RestoListaArgsFormales> | ε
<ArgFormal> ::= <Tipo> <_TipoParametricoOpcional> idMetVar
<BloqueOpcional> ::= <Bloque> | ;
<Bloque> ::= { <ListaSentencias> }
<ListaSentencias> ::= <Sentencia> <ListaSentencias> | ε
<Sentencia> ::= ; | idClase <_DecisorExpresionIdClase> | <Expresion> ;
    <_VarLocalClasica> ; | <VarLocal> ; | <ReturnStatement> ; | <Bloque>
    | <IfStatement> | <WhileStatement> | <_ForStatement>
<_DecisorExpresionIdClase> ::= . idMetVar <ArgsActuales> |
    <_RestoVarLocalClasica> | <ExpresionCompuesta>
    <_RestoVarLocalClasica> <_RestoExpresion> <_OperadorTernario> ;
<VarLocal> ::= <_InicioVarLocal> <_RestoVarLocal>
<_InicioVarLocal> ::= var idMetVar
<_RestoVarLocal> ::= = <ExpresionCompuesta> <_OperadorTernario>
<_OperadorTernario> ::= ? <Expresion> : <Expresion> | ε
<ReturnStatement> ::= return <ExpresionOpcional>
<ExpresionOpcional> ::= <Expresion> | ε
<IfStatement> ::= if ( <Expresion> ) <Sentencia> <_RestoIfStatement>
<_RestoIfStatement> ::= else <Sentencia> | ε
<WhileStatement> ::= while ( <Expresion> ) <Sentencia>
<Expresion> ::= <ExpresionCompuesta> <_RestoExpresion>
<_RestoExpresion> ::= <OperadorAsignacion> <ExpresionCompuesta> | ε
<OperadorAsignacion> ::= =
<ExpresionCompuesta> ::= <ExpresionBasica> <_RestoExpresionCompuesta>
<_RestoExpresionCompuesta> ::= <OperadorBinario> <ExpresionBasica>
    <_RestoExpresionCompuesta> | ε
<OperadorBinario> ::= || | && | == | != | < | > | <= | >= | + | * | / |
    - | %
<ExpresionBasica> ::= <OperadorUnario> <Operando> | <Operando>
<OperadorUnario> ::= + | ++ | - | -- | !
<Operando> ::= <Primitivo> | <Referencia>
<Primitivo> ::= true | false | intLiteral | charLiteral | null
<Referencia> ::= <Primario> <_RestoReferencia>

```

```

<_RestoReferencia> ::= <_Encadenado> <_RestoReferencia> | ε
<_Encadenado> ::= . idMetVar <_RestoEncadenado>
<_RestoEncadenado> ::= <ArgsActuales> | ε
<Primario> ::= this | stringLiteral | <LlamadaConstructor> |
    idMetVar <_RestoLlamadaMetodo> | <LlamadaMetodoEstatico> |
    <ExpresionParentizada>
<_RestoLlamadaMetodo> ::= <ArgsActuales> | ε
<LlamadaConstructor> ::=
    new idClase <_TipoParametricoInstanciacion> <ArgsActuales>
<_TipoParametricoInstanciacion> ::= < <_ParametrosGenericidad> | ε
<_ParametrosGenericidad> ::= > | idClase >
<LlamadaMetodoEstatico> ::= idClase . idMetVar <ArgsActuales>
<ExpresionParentizada> ::= ( <Expresion> )
<ArgsActuales> ::= ( <ListaExpsOpcional> )
<ListaExpsOpcional> ::= <ListaExps> | ε
<ListaExps> ::= <Expresion> <_RestoListaExps>
<_RestoListaExps> ::= , <ListaExps> | ε
<_ForStatement> ::= for ( <_RestoFor>
<_RestoFor> ::= <Tipo> idMetVar <_DecisorForTipo> |
    <_InicioVarLocal> <_DecisorForVarLocal> | <_ForEstandar>
<_DecisorForTipo> ::= : idMetVar ) <BloqueOpcional> |
    ; <ExpresionOpcional> ; <_IncrementoOpcional> ) <BloqueOpcional> |
    <_RestoVarLocal> ; <ExpresionOpcional> ; <_IncrementoOpcional> )
    <BloqueOpcional>
<_DecisorForVarLocal> ::= : idMetVar ) <BloqueOpcional> |
    <_RestoVarLocal> ; <ExpresionOpcional> ; <_IncrementoOpcional> )
    <BloqueOpcional>
<_ForEstandar> ::= <_DeclaracionOpcional> ; <ExpresionOpcional> ;
    <_IncrementoOpcional> ) <BloqueOpcional>
<_RestoIdMetVarForEstandar> ::= = <ExpresionCompuesta> |
    <_OperadorUnarioModificador>
<_IncrementoOpcional> ::= idMetVar <_RestoIdMetVarForEstandar> |
    <ExpresionBasica> | ε
<_OperadorUnarioModificador> ::= ++ | --
<_DeclaracionTipoOpcional> ::= <Tipo> <_TipoParametricoOpcional> | ε

```

```

<_DeclaracionOpcional> ::=
    <_DeclaracionTipoOpcional> idMetVar <_AsignacionOpcional> | ε
<_VarLocalClasica> ::=
    <Tipo> <_TipoParametricoOpcional> idMetVar
    <_RestoVarLocalClasica> <_AsignacionOpcional>
<_RestoVarLocalClasica> ::= , idMetVar <_RestoVarLocalClasica> | ε
<_AsignacionOpcional> ::=
    <OperadorAsignacion> <ExpresionCompuesta> <_OperadorTernario> | ε

```

Logros

Se intentaron alcanzar los siguientes logros:

Logros de la etapa 1

- manejador de archivos eficiente,
- unicodes,
- multi detección de errores léxicos.

Dentro del método `main`, se encuentra comentada la línea `lexicalAnalysis()`; descomentándola, corre el analizador léxico propuesto en la etapa 1. Debe comentarse la línea siguiente `syntacticAnalysis()`; para evitar mostrar errores no relativos al análisis léxico.

```

public static void main(String[] args) {
    initialize();

    if (args.length == 1) {
        String fileName = args[0];
        openFile(fileName);
        //lexicalAnalysis();
        syntacticAnalysis();
        closeFile();
    }
    else {
        System.out.println(GenericErrorMessage.MISUSE_ERROR);
    }
}

```

Logros de la etapa 2

- variables locales clásicas E2,
- entrega anticipada sintáctica,
- imbatibilidad sintáctica,
- genericidad E2,
- operador ternario E2,

- visibilidad mejorada E2,
- fors! E2,
- interfaces E2,
- atributos inicializados,
- interfaces genéricas E2

Los logros **recuperación de errores modo pánico** e **implementación avanzada** no fueron implementados.

Repositorio

<https://github.com/ludmilapolygin/compiladorMiniJava/tree/Etapa2>